Aplicación web con MariaDB — Práctica ABD 26/27

1. Qué he realizado

He preparado una aplicación web en Python con Flask que se conecta a mi servidor MariaDB. Desde la web puedo iniciar sesión con una cuenta de la base de datos, ver sus tablas permitidas y consultar registros. También he demostrado una inyección SQL en un buscador y la he corregido.

He utilizado MariaDB. La aplicación también puede adaptarse a MySQL con PyMySQL, manteniendo el mismo planteamiento; tendría que revisar la configuración y las opciones propias del servidor. No son el mismo producto. En esta memoria documento MariaDB, que es el que he usado.

2. Mi entorno

Equipo	Uso	Dirección
Sergio-PC	Anfitrión de LXC y Virt-Manager	Accedo a la web desde este equipo
app-web	Contenedor Debian 13 con Flask	10.0.3.46
sgbd-vm	Máquina virtual de Virt-Manager con MariaDB	192.168.122.22

Dentro de la máquina virtual, el nombre del sistema es `bd-sergio`. El contenedor está conectado a `lxcbr0` (10.0.3.1) y la máquina virtual a la red de `virbr0` (192.168.122.1).

La base se llama `practica_web` y MariaDB escucha en el puerto 3306. Por tanto, la aplicación y el servidor de base de datos están en sistemas separados, como pide la práctica.

La dirección de mi aplicación es:

```
http://10.0.3.46:5000
```

Las IP son las obtenidas en mi instalación. Si vuelvo a crear las máquinas, compruebo la dirección asignada antes de conectar.

3. Creación del contenedor LXC

En mi PC anfitrión

Con LXC clásico instalado, creo el contenedor Debian 13:

```
sudo lxc-create -n app-web -t download -- --dist debian --release trixie --arch
amd64
```

El fichero generado está en `/var/lib/lxc/app-web/config`. Compruebo su contenido:

```
sudo cat /var/lib/lxc/app-web/config
```

Las líneas de configuración de mi contenedor son:

```
lxc.include = /usr/share/lxc/config/common.conf
lxc.arch = linux64
lxc.apparmor.profile = generated
lxc.apparmor.allow_nesting = 1
lxc.rootfs.path = dir:/var/lib/lxc/app-web/rootfs
lxc.uts.name = app-web

lxc.net.0.type = veth
lxc.net.0.link = lxcbr0
lxc.net.0.flags = up
```

He mantenido el puente `lxcbr0`, que ya estaba configurado en mi anfitrión. No necesito modificar los demás puentes de mis prácticas. El permiso de nesting aparece en mi configuración; la aplicación Flask no depende de él.

Arranco el contenedor y entro:

```
sudo lxc-start -n app-web
sudo lxc-attach -n app-web
```

Dentro del contenedor

Compruebo la versión y dirección:

```
cat /etc/os-release
ip -br addr
```

En mi caso aparece Debian GNU/Linux 13 (trixie) y la interfaz `eth0` tiene `10.0.3.46/24`.

4. Preparación de MariaDB en sgbd-vm

Esta parte la realizo en la máquina virtual `sgbd-vm`, no dentro del contenedor web. Parto de MariaDB ya instalado en la parte de servidores de la práctica.

4.1 Permitir conexiones remotas

Edito el fichero:

```
sudo nano /etc/mysql/mariadb.conf.d/50-server.cnf
```

La directiva queda así:

```
bind-address = 0.0.0.0
```

Con esto MariaDB escucha en las interfaces de red, no solamente en localhost. Mantengo el servidor dentro de mi laboratorio; no abro el puerto a Internet.

Aplico el cambio y entro al servidor:

```
sudo systemctl restart mariadb
sudo mariadb
```

Compruebo la configuración:

```
SHOW VARIABLES LIKE 'bind_address';
```

El resultado es `0.0.0.0`.

4.2 Crear la base y las tablas

Para reproducir la base desde cero, ejecuto este SQL dentro de MariaDB. No lo vuelvo a ejecutar sobre la base ya preparada, porque las tablas y registros ya existen.

```sql
CREATE DATABASE practica_web
  CHARACTER SET utf8mb4 COLLATE utf8mb4_uca1400_ai_ci;
USE practica_web;

CREATE TABLE clientes (
  id INT NOT NULL AUTO_INCREMENT,
  nombre VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL,
  PRIMARY KEY (id),
  UNIQUE KEY email (email)
) ENGINE=InnoDB;

CREATE TABLE notas (
  id INT NOT NULL AUTO_INCREMENT,
  titulo VARCHAR(100) NOT NULL,
  contenido VARCHAR(500) NOT NULL,
  interna TINYINT(1) NOT NULL DEFAULT 0,
  PRIMARY KEY (id)
) ENGINE=InnoDB;

CREATE TABLE pedidos (
  id INT NOT NULL AUTO_INCREMENT,
  cliente_id INT NOT NULL,
  producto VARCHAR(100) NOT NULL,
  importe DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (id),
  KEY cliente_id (cliente_id),
  CONSTRAINT pedidos_ibfk_1 FOREIGN KEY (cliente_id)
    REFERENCES clientes(id)
) ENGINE=InnoDB;

CREATE TABLE notas_privadas (
  id INT NOT NULL AUTO_INCREMENT,
  nota VARCHAR(200) NOT NULL,
  PRIMARY KEY (id)
) ENGINE=InnoDB;
```

La opción de cotejamiento de esta base pertenece a MariaDB. Si adapto el proyecto a MySQL, uso un cotejamiento compatible con la versión instalada.

4.3 Introducir los datos de prueba

```
INSERT INTO clientes (nombre, email) VALUES
('Sergio', 'sergio@example.com'),
('Ana', 'ana@example.com'),
('Luis', 'luis@example.com');

INSERT INTO notas (titulo, contenido, interna) VALUES
('Horario', 'Atención de 9:00 a 14:00', 0),
('Mantenimiento', 'Reinicio previsto el viernes', 0),
('Presupuesto pendiente', 'Propuesta interna de 4500 euros', 1);

INSERT INTO pedidos (cliente_id, producto, importe) VALUES
(1, 'Teclado', 39.90),
(2, 'Monitor', 149.50),
(3, 'Ratón', 19.90);

INSERT INTO notas_privadas (nota) VALUES
('Dato de prueba reservado para la demostración');
```

He creado cuatro tablas. En `notas`, el valor `interna = 1` identifica la nota que oculto en la web a los usuarios distintos de admin.

4.4 Crear los usuarios y asignar permisos

Para el laboratorio utilizo estas cuentas y contraseñas de prueba. No son contraseñas para un servicio publicado.

```
CREATE USER 'lectura'@'%' IDENTIFIED BY 'lectura1223';
CREATE USER 'admin'@'%' IDENTIFIED BY 'admin1223';

GRANT SELECT ON practica_web.clientes TO 'lectura'@'%';
GRANT SELECT ON practica_web.notas TO 'lectura'@'%';

GRANT SELECT ON practica_web.* TO 'admin'@'%';
GRANT SELECT ON practica_web.notas_privadas TO 'admin'@'%';
```

El último permiso de admin es redundante porque ya tiene SELECT sobre toda la base, pero aparece así en mis permisos reales.

Compruebo el resultado:

```
SHOW GRANTS FOR 'lectura'@'%';
SHOW GRANTS FOR 'admin'@'%';
```

Usuario	Tablas que puede consultar
lectura	clientes y notas
admin	clientes, notas, pedidos y notas_privadas

Aunque se llama admin, esta cuenta tiene permisos SELECT en esta base: el nombre no la convierte en administrador global. El host `%` permite conexiones desde distintos orígenes si la red las permite; lo he usado dentro del laboratorio.

La cuenta lectura puede leer la tabla `notas` completa desde SQL. La restricción que oculta las filas internas la añado en la aplicación. Esta distinción es importante para la demostración de inyección.

Salgo de MariaDB:

```
exit;
```

5. Preparación del proyecto Python

Vuelvo al contenedor desde mi PC:

```
sudo lxc-attach -n app-web
```

Dentro instalo lo necesario:

```
apt update
apt install -y python3 python3-venv python3-pip nano default-mysql-client
mkdir -p /root/app-web/templates
cd /root/app-web
python3 -m venv .venv
source .venv/bin/activate
pip install Flask Flask-Session cachelib PyMySQL
pip freeze > requirements.txt
```

El prefijo `(.venv)` me indica que estoy usando el entorno del proyecto. Las dependencias quedan recogidas en `requirements.txt`, por lo que puedo reinstalarlas con `pip install -r requirements.txt`.

Compruebo la conexión remota desde el contenedor:

```
mariadb -h 192.168.122.22 -P 3306 -u lectura -p practica_web
```

Introduzco la contraseña cuando la pide y ejecuto:

```
SHOW TABLES;
exit;
```

Deben aparecer `clientes` y `notas`. Esta conexión confirma que el contenedor puede autenticarse en el servidor remoto.

6. Creación de los archivos

Creo cada archivo con `nano`. Guardo con `Ctrl+O`, pulso Enter y salgo con `Ctrl+X`. Todos los comandos de esta sección se ejecutan dentro de `/root/app-web`, en el contenedor.

6.1 config.py

```
nano config.py
```

Contenido:

```
MYSQL_HOST = "192.168.122.22"
MYSQL_PORT = 3306
MYSQL_DATABASE = "practica_web"
```

Aquí guardo dónde está la base. El usuario y la contraseña se solicitan en la pantalla de acceso.

6.2 Clave privada de la aplicación

No incluyo mi clave secreta en la documentación. Para que el proyecto funcione sin tener que escribirla en el código, la genero una vez en un archivo local:

```
python3 - <<'PY'
from pathlib import Path
import secrets
clave = Path('.secret_key')
if not clave.exists():
    clave.write_text(secrets.token_hex(32))
clave.chmod(0o600)
PY
```

Este pequeño ajuste evita publicar la clave que tenía escrita en `app.py`. La lógica de la aplicación es la misma.

6.3 app.py

```
nano app.py
```

Este es el contenido completo con la búsqueda corregida:

```python
from flask import Flask, request, session, redirect, url_for, render_template,
flash
import pymysql
import config
from pathlib import Path
from flask_session import Session
from cachelib.file import FileSystemCache

app = Flask(__name__)
app.secret_key = Path("/root/app-web/.secret_key").read_text().strip()
app.config["SESSION_TYPE"] = "cachelib"
app.config["SESSION_CACHELIB"] = FileSystemCache(
    cache_dir="/root/app-web/.sessions", threshold=100
)
Session(app)


def conectar(usuario, password):
    return pymysql.connect(
        host=config.MYSQL_HOST, port=config.MYSQL_PORT,
        user=usuario, password=password, database=config.MYSQL_DATABASE,
        cursorclass=pymysql.cursors.DictCursor, connect_timeout=5
    )


@app.route("/", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        usuario = request.form.get("usuario", "").strip()
        password = request.form.get("password", "")
        try:
            conexion = conectar(usuario, password)
            conexion.close()
            session.clear()
            session["usuario"] = usuario
            session["password"] = password
            return redirect(url_for("tablas"))
        except pymysql.MySQLError:
            flash("Usuario o contraseña incorrectos.")
    return render_template("login.html")


@app.route("/tablas")
def tablas():
    if "usuario" not in session:
        return redirect(url_for("login"))
```

```
    conexion = conectar(session["usuario"], session["password"])
    try:
        with conexion.cursor() as cursor:
            cursor.execute("SHOW TABLES")
            nombres = [next(iter(fila.values())) for fila in cursor.fetchall()]
    finally:
        conexion.close()
    return render_template("tablas.html", tablas=nombres)


@app.route("/tabla/<nombre>")
def ver_tabla(nombre):
    if "usuario" not in session:
        return redirect(url_for("login"))
    conexion = conectar(session["usuario"], session["password"])
    try:
        with conexion.cursor() as cursor:
            cursor.execute("SHOW TABLES")
            permitidas = [next(iter(fila.values())) for fila in
cursor.fetchall()]
            if nombre not in permitidas:
                flash("Esa tabla no está disponible.")
                return redirect(url_for("tablas"))
            if nombre == "notas" and session["usuario"] != "admin":
                cursor.execute(
                    "SELECT id, titulo, contenido FROM notas "
                    "WHERE interna = 0 LIMIT 100"
                )
            else:
                cursor.execute(f"SELECT * FROM `{nombre}` LIMIT 100")
            registros = cursor.fetchall()
            columnas = [columna[0] for columna in cursor.description]
    finally:
        conexion.close()
    return render_template(
        "tabla.html", nombre=nombre, columnas=columnas, registros=registros
    )


@app.route("/buscar-notas", methods=["GET", "POST"])
def buscar_notas():
    if "usuario" not in session:
        return redirect(url_for("login"))
    resultados = []
    termino = ""
    if request.method == "POST":
```

```
        termino = request.form.get("termino", "")
        conexion = conectar(session["usuario"], session["password"])
        try:
            with conexion.cursor() as cursor:
                consulta = (
                    "SELECT id, titulo, contenido FROM notas "
                    "WHERE titulo LIKE %s"
                )
                if session["usuario"] != "admin":
                    consulta += " AND interna = 0"
                consulta += " LIMIT 100"
                cursor.execute(consulta, (f"%{termino}%",))
                resultados = cursor.fetchall()
        except pymysql.MySQLError:
            flash("No se pudo realizar la búsqueda.")
        finally:
            conexion.close()
    return render_template(
        "buscar_notas.html", termino=termino, resultados=resultados,
        buscado=request.method == "POST"
    )


@app.route("/logout")
def logout():
    session.clear()
    return redirect(url_for("login"))


if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000, debug=False)
```

Resumen de las funciones

Función	Qué hace
conectar	Abre la conexión con el servidor remoto usando las credenciales recibidas.
login	Prueba las credenciales, inicia la sesión y lleva a las tablas.
tablas	Obtiene las tablas visibles para esa cuenta con SHOW TABLES.
ver_tabla	Comprueba la tabla solicitada y muestra sus registros.
buscar_notas	Busca por título y aplica el filtro de notas internas.
logout	Borra la sesión y vuelve al acceso.

Uso `DictCursor` para obtener filas con nombres de columnas. `render_template` envía los datos a las páginas HTML. Las sesiones se guardan en el servidor mediante Flask-Session y CacheLib.

En esta práctica guardo temporalmente las credenciales de la base en la sesión del servidor para reconectar como ese usuario. Es una simplificación del laboratorio, no un diseño para publicar la aplicación.

6.4 templates/base.html

Contiene el diseño común, el menú y los mensajes.

```
nano templates/base.html
```

```
<!doctype html>
<html lang="es">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Panel MariaDB</title>
<style>
body { margin:0; font-family:Arial,sans-serif; color:#243247;
background:#f1f5f9; }
header { background:#17324d; color:white; padding:20px; }
nav { display:flex; gap:20px; }
nav a { color:white; }
main { max-width:900px; margin:30px auto; padding:15px; }
.card { background:white; padding:24px; border-radius:10px; }
input,button { padding:10px; font-size:16px; }
button { background:#176b87; color:white; border:0; cursor:pointer; }
table { width:100%; border-collapse:collapse; margin-top:20px; }
th,td { padding:12px; text-align:left; border-bottom:1px solid #ddd; }
th,.notice { background:#e9f2f8; }
.notice,.error { padding:12px; }
.error { background:#fff0f0; color:#9d2424; }
a { color:#176b87; }
</style>
</head>
<body>
<header>
<h1>Panel de consulta MariaDB</h1>
{% if session.get("usuario") %}
<nav>
<a href="{{ url_for('tablas') }}">Tablas</a>
<a href="{{ url_for('buscar_notas') }}">Buscar notas</a>
<a href="{{ url_for('logout') }}">Cerrar sesión</a>
</nav>
{% endif %}
</header>
<main>
{% for mensaje in get_flashed_messages() %}<p class="error">{{ mensaje }}</p>{%
endfor %}
<section class="card">{% block contenido %}{% endblock %}</section>
</main>
</body>
</html>
```

6.5 templates/login.html

Contiene el formulario de autenticación.

```
nano templates/login.html
```

```
{% extends "base.html" %}
{% block contenido %}
<h2>Iniciar sesión</h2>
<p>Accede con una cuenta MariaDB.</p>
<form method="post">
<label for="usuario">Usuario</label><br><br>
<input id="usuario" name="usuario" autocomplete="username" required><br><br>
<label for="password">Contraseña</label><br><br>
<input id="password" type="password" name="password" autocomplete="current-
password" required><br><br>
<button type="submit">Entrar</button>
</form>
{% endblock %}
```

6.6 templates/tablas.html

Recorre las tablas permitidas y crea sus enlaces.

```
nano templates/tablas.html
```

```
{% extends "base.html" %}
{% block contenido %}
<h2>Tablas accesibles</h2>
<p class="notice">Usuario conectado: {{ session["usuario"] }}</p>
<ul>
{% for tabla in tablas %}
<li><a href="{{ url_for('ver_tabla', nombre=tabla) }}">{{ tabla }}</a></li>
{% else %}<li>No hay tablas disponibles.</li>{% endfor %}
</ul>
{% endblock %}
```

6.7 templates/tabla.html

Muestra las columnas y registros de la tabla elegida.

```
nano templates/tabla.html
```

```
{% extends "base.html" %}
{% block contenido %}
<h2>Registros de {{ nombre }}</h2>
{% if nombre == "notas" %}
{% if session["usuario"] == "admin" %}
<p class="notice">Puedes consultar las notas compartidas e internas.</p>
{% else %}<p class="notice">Solo se muestran las notas compartidas.</p>{% endif
%}
{% endif %}
{% if registros %}
<table>
<tr>{% for columna in columnas %}<th>{{ columna }}</th>{% endfor %}</tr>
{% for registro in registros %}
<tr>{% for columna in columnas %}<td>{{ registro[columna] }}</td>{% endfor %}
</tr>
{% endfor %}
</table>
{% else %}<p>No hay registros disponibles.</p>{% endif %}
{% endblock %}
```

6.8 templates/buscar_notas.html

Contiene el formulario de búsqueda y sus resultados.

```
nano templates/buscar_notas.html
```

```
{% extends "base.html" %}
{% block contenido %}
<h2>Buscar notas de trabajo</h2>
<p class="notice">
Usuario: {{ session["usuario"] }}.
{% if session["usuario"] == "admin" %}
Puedes consultar las notas compartidas e internas.
{% else %}Solo puedes consultar notas compartidas.{% endif %}
</p>
<form method="post">
<label for="termino">Título de la nota</label><br><br>
<input id="termino" name="termino" value="{{ termino }}" placeholder="Por
ejemplo: Horario" required>
<button type="submit">Buscar</button>
</form>
{% if resultados %}
<table>
<tr><th>ID</th><th>Título</th><th>Contenido</th></tr>
{% for nota in resultados %}
<tr><td>{{ nota.id }}</td><td>{{ nota.titulo }}</td><td>{{ nota.contenido }}
</td></tr>
{% endfor %}
</table>
{% elif buscado %}<p>No se encontraron coincidencias.</p>{% endif %}
{% endblock %}
```

7. Arrancar y comprobar la aplicación

Dentro del contenedor:

```
cd /root/app-web
source .venv/bin/activate
python3 -m py_compile app.py
python3 app.py
```

Abro desde el navegador de mi PC:

```
http://10.0.3.46:5000
```

Prueba con lectura

Inicio sesión con `lectura` y su contraseña de laboratorio. Aparecen `clientes` y `notas`. Al abrir `notas` solo aparecen Horario y Mantenimiento, porque la web añade `WHERE interna =`

`0`.

Prueba con admin

Cierro sesión y entro con `admin`. Aparecen las cuatro tablas. En `notas` también aparece Presupuesto pendiente.

La diferencia en las tablas disponibles depende de los permisos de MariaDB; la diferencia entre filas compartidas e internas de `notas` depende del filtro de la aplicación.

8. Preparar la versión vulnerable

La versión anterior de `app.py` usa parámetros y está corregida. Primero guardo esa copia:

```
cp app.py app_seguro.py
```

Para reproducir el fallo del laboratorio, edito `app.py`:

```
nano app.py
```

Dentro de `buscar_notas`, sustituyo solo el bloque que construye y ejecuta la consulta por este, conservando la indentación:

```
                consulta = (
                    "SELECT id, titulo, contenido FROM notas "
                    "WHERE titulo LIKE '%" + termino + "%'")

                if session["usuario"] != "admin":
                    consulta += " AND interna = 0"

                cursor.execute(consulta + " LIMIT 100")
```

Dejo `resultados = cursor.fetchall()` justo después, como estaba. Guardo la versión vulnerable:

```
python3 -m py_compile app.py
cp app.py app_vulnerable.py
```

9. Demostración de SQL Injection

Realizo esta prueba solo en mi aplicación del laboratorio y con los datos ficticios de la práctica.

Arranco la versión vulnerable y entro como `lectura`:

```
python3 app.py
```

En Buscar notas hago estas pruebas:

Búsqueda	Resultado vulnerable
Horario	Devuelve la nota Horario.
Presupuesto	No devuelve coincidencias.
NoExiste' OR 1=1 #	Devuelve también la nota interna.

La entrada de demostración es:

```
NoExiste' OR 1=1 #
```

La consulta queda así:

```
SELECT id, titulo, contenido FROM notas
WHERE titulo LIKE '%NoExiste' OR 1=1 #%' AND interna = 0 LIMIT 100
```

La condición `OR 1=1` es verdadera y `#` comenta el final de esa línea. Así deja de aplicarse `AND interna = 0`.

El error está en concatenar el término del formulario dentro del SQL. No he convertido lectura en administrador ni cambiado sus permisos: he evitado el filtro de la aplicación.

10. Corrección y comprobación

La solución consiste en separar la consulta de los valores. El bloque corregido de `buscar_notas` es:

```
            consulta = (
                "SELECT id, titulo, contenido FROM notas "
                "WHERE titulo LIKE %s"
            )

            if session["usuario"] != "admin":
                consulta += " AND interna = 0"

            consulta += " LIMIT 100"
            cursor.execute(consulta, (f"%{termino}%",))
```

El marcador `%s` no lleva comillas. El valor se pasa como segundo argumento de `execute`, dentro de una tupla. Los porcentajes del valor permiten buscar coincidencias parciales. El texto introducido ya no puede convertirse en instrucciones SQL.

Como ya tengo esa versión guardada, detengo Flask con `Ctrl+C` y la activo:

```
cp app_seguro.py app.py
python3 -m py_compile app.py && python3 app.py
```

Repito las pruebas como lectura:

Búsqueda	Resultado corregido
Horario	Devuelve la nota Horario.
Presupuesto	No devuelve coincidencias.
NoExiste' OR 1=1 #	No devuelve coincidencias ni revela notas internas.

Después compruebo con admin que buscar Presupuesto sigue funcionando. Así verifico que la corrección no rompe el acceso permitido.

11. Cambiar de versión durante el vídeo

Siempre detengo Flask con `Ctrl+C` antes de cambiar el archivo.

Para mostrar el fallo:

```
cp app_vulnerable.py app.py
python3 app.py
```

Para mostrar la solución:

```
cp app_seguro.py app.py
python3 app.py
```

`cp` copia del primer nombre al segundo. No ejecuto `cp app.py app_seguro.py` mientras app.py sea vulnerable, porque sobrescribiría la copia corregida.

Al terminar dejo activa la versión segura.

12. Volver a entrar y errores que resolví

Si salgo del contenedor

Desde mi PC:

```
sudo lxc-attach -n app-web
```

Dentro:

```
cd /root/app-web
source .venv/bin/activate
python3 app.py
```

Error de tabulaciones

Al pegar código apareció `TabError: inconsistent use of tabs and spaces in indentation`. Lo resolví reescribiendo el bloque con espacios y comprobando antes de arrancar:

```
python3 -m py_compile app.py
```

Una búsqueda no encuentra resultados

El buscador busca títulos de notas, no nombres de clientes. Buscar Sergio no encuentra una nota solo porque exista un cliente con ese nombre.

13. Archivos que conservo para la entrega

```
app-web/
├── app.py
├── app_seguro.py
├── app_vulnerable.py
├── config.py
├── requirements.txt
└── templates/
    ├── base.html
    ├── login.html
    ├── tablas.html
    ├── tabla.html
    └── buscar_notas.html
```

Los archivos `buscar.html` y `catalogo.html` que quedaron de las primeras pruebas no son necesarios en esta versión: sus rutas y enlaces ya no se utilizan.

No entrego la clave `.secret_key`, los archivos de sesión `.sessions` ni el entorno `.venv`. La clave se vuelve a generar al preparar otra instalación siguiendo el paso 6.2.

14. Resultado

He separado el cliente web y el servidor MariaDB, he comprobado la autenticación y el acceso a tablas según los permisos, y he mostrado registros desde la aplicación. También he reproducido una inyección SQL en el buscador y comprobado que una consulta parametrizada evita ese ataque.